

NED UNIVERSITY OF ENGINEERING AND TECHNOLOGY



PROJECT REPORT ON

**Automated Extraction and Exploratory Analysis of
Court Judgment Data (Feature Engineering and
Machine Learning Model Development Report)**

BY

NAME: SYED MOHAMMAD HUZAIFA

ROLL No: CTAI-022 (2023 Batch)

Under The Supervision Of

DR.WASEEMULLAH NAZIR

Associate Professor

Department Of Computer Science and Information Technology

Contents

Under The Supervision Of	1
1. Introduction	3
2. Why New Features Were Added	3
3. Newly Engineered Features – Explanation & Purpose	4
Feature 6: Hearing Gap Days (Case Duration Indicator)	4
Action Taken.....	4
Rationale.....	4
Feature 7: Most Common Legal Sections (Top Section Frequency)	5
Action Taken.....	5
Rationale.....	5
Feature 8: Legal Category Classification	6
Action Taken.....	6
Rationale.....	6
Feature 9: Normalized Complexity-Based Ratios	7
Action Taken.....	7
Rationale.....	7
4. Model Development Explanation.....	8
4.1 Random Forest (Baseline Model).....	8
4.2 XGBoost (Boosting Approach).....	9
4.3 Linear Models (Final Attempt)	10
7. Conclusion	12

1. Introduction

Legal judgment documents contain rich but highly unstructured information, making computational analysis and prediction extremely difficult. After extracting essential metadata from court judgment PDFs—such as case details, legal sections, parties involved, precedents, and judgment outcomes—a detailed Exploratory Data Analysis (EDA) revealed several important trends. However, it also became clear that the raw dataset lacked sufficient signal for machine learning models to learn from.

The initial round of feature engineering provided basic text lengths, section counts, and simple metadata transformations, but these features were not enough to support meaningful predictive modeling.

To overcome this, a more advanced round of targeted feature engineering was performed, focusing on structural complexity, case duration, legal categorization, and normalized text metrics. These additions were essential for training machine learning models such as Random Forest, XGBoost, and Linear classifiers.

Despite introducing these features, the small dataset size (only 55 cases) remained the primary limitation. Yet the new features significantly strengthened the dataset and created a more suitable foundation for future modeling on larger data.

2. Why New Features Were Added

During early experimentation, all three machine learning models failed to produce meaningful predictions:

- The dataset was too small for tree-based models (Random Forest, XGBoost).
- Linear models struggled due to class imbalance and lack of discriminative features.
- The initial engineered features did not capture the complexity of legal judgments.

Therefore, additional features were introduced to capture:

- **Case complexity**
- **Textual depth**
- **Legal diversity**
- **Temporal characteristics**
- **Normalized ratios** that provide more stable signals to ML models

These features enhanced the dataset's richness and attempted to provide models with meaningful patterns.

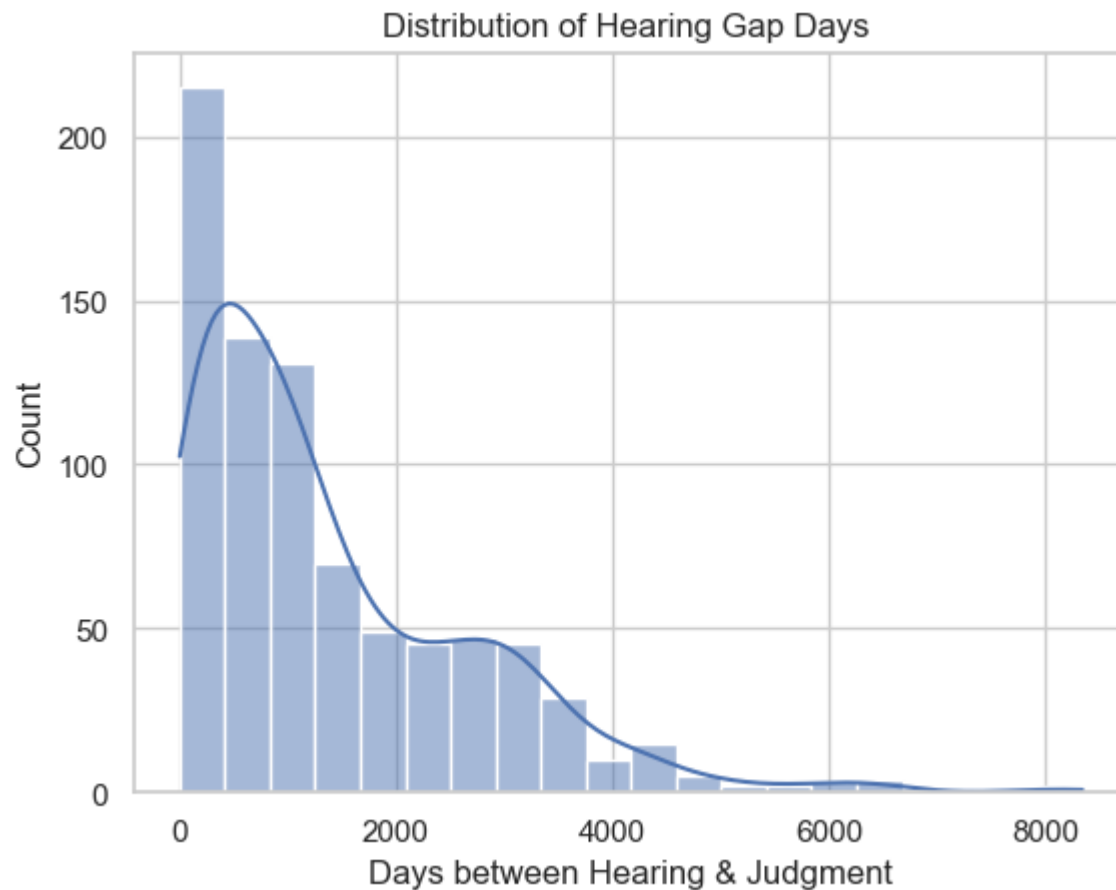
3. Newly Engineered Features – Explanation & Purpose

Below are the new features added after EDA, along with why they were needed.

Feature 6: Hearing Gap Days (Case Duration Indicator)

Action Taken

hearing_gap_days = Days between hearing date and judgment date.



Rationale

Case duration is a proxy for:

- Judicial workload
- Case urgency

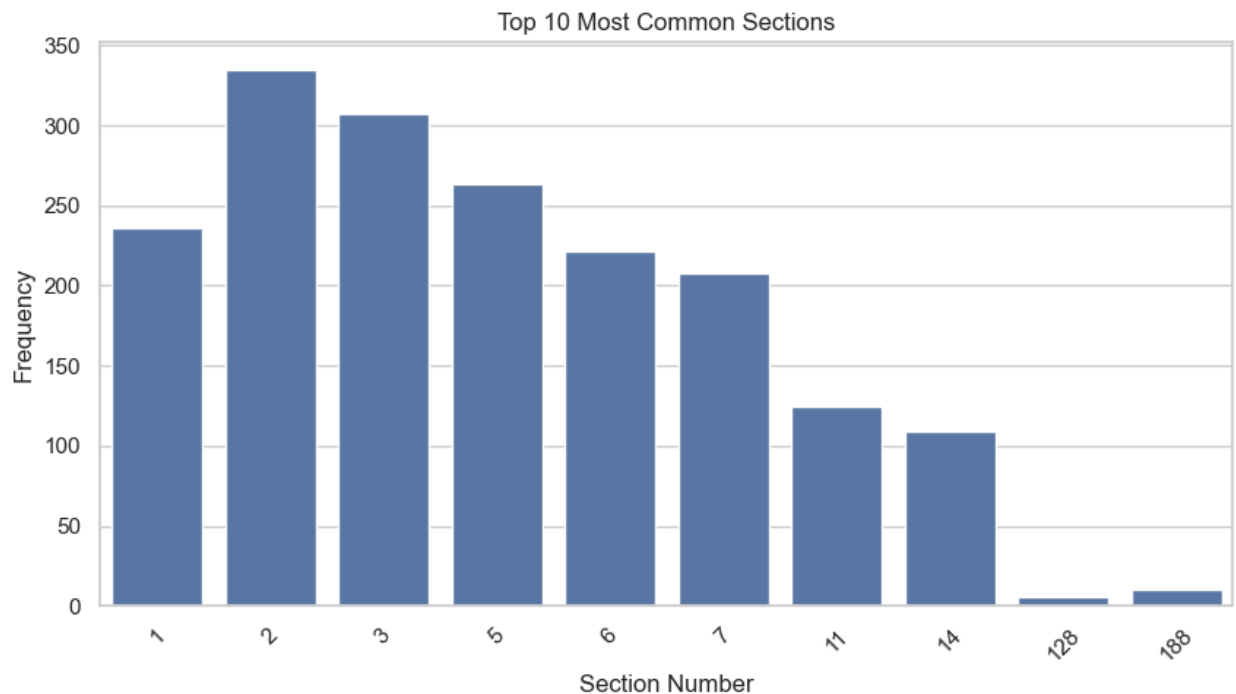
- Complexity or requirement for deeper evaluation

More complex cases often take longer to decide, making this a valuable predictive signal.

Feature 7: Most Common Legal Sections (Top Section Frequency)

Action Taken

A frequency distribution of all legal sections across cases was generated.



Rationale

Certain sections (e.g., constitutional, criminal, or civil statutes) often correlate with specific types of outcomes.

Identifying section frequency helps:

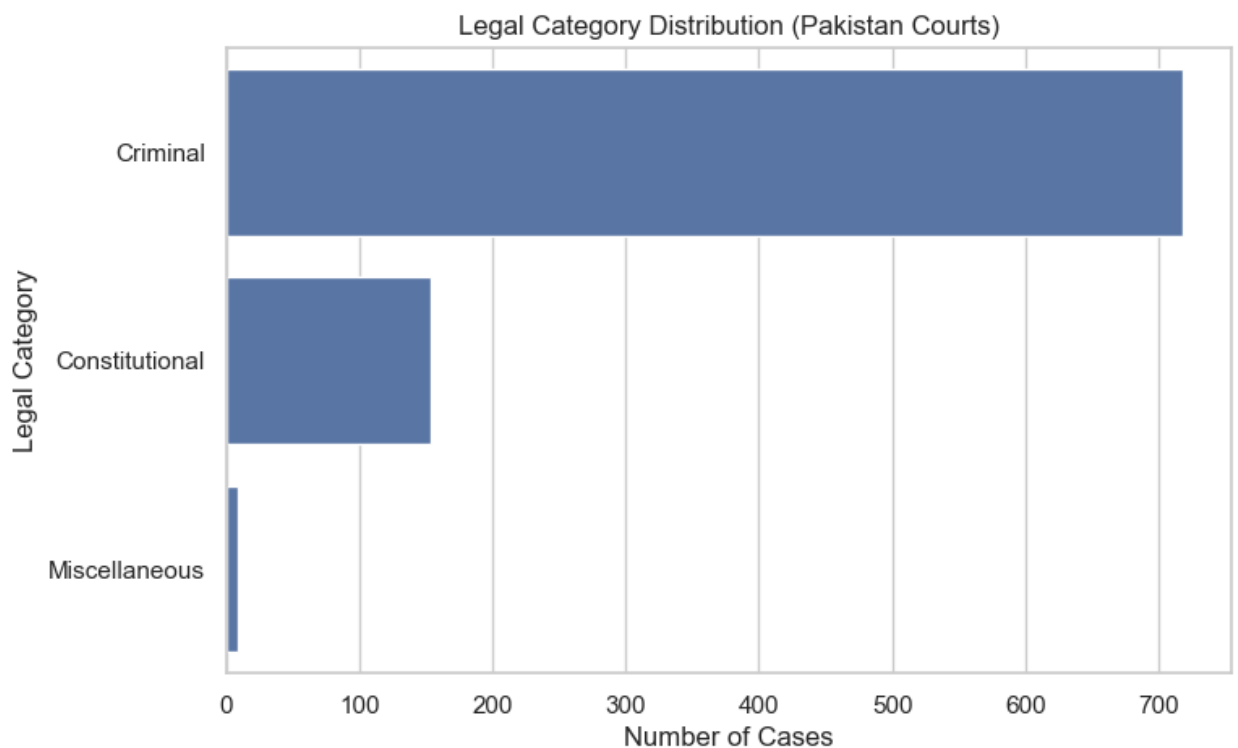
- Understand legal focus areas
 - Group cases by context
 - Build categorical signals for modeling
-

Feature 8: Legal Category Classification

Action Taken

Cases were mapped to broad legal categories:

- Criminal
- Civil
- Constitutional
- Service
- Taxation
- Others



Rationale

Outcome tendencies differ across legal domains. For example:

- Criminal appeals often follow stricter standards
- Constitutional petitions may involve rights-based reasoning

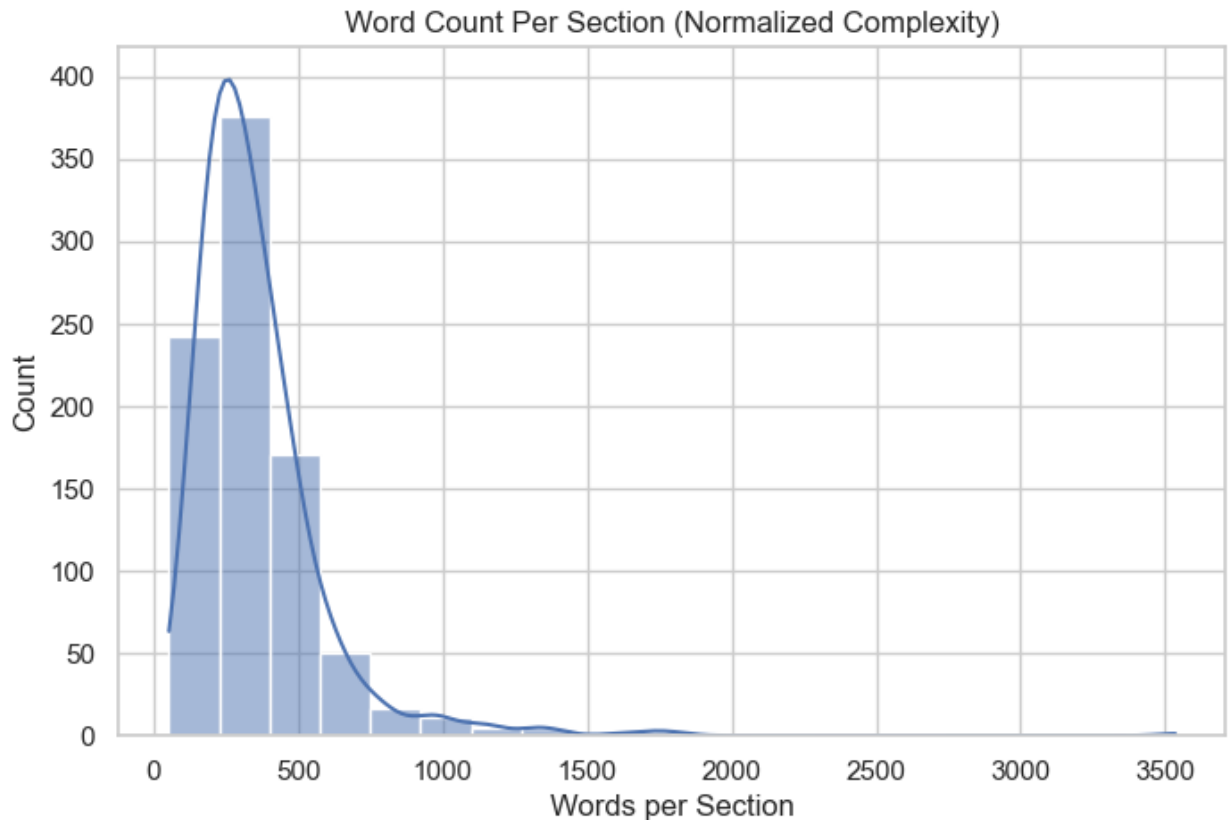
This feature introduces domain-aware segmentation for the ML model.

Feature 9: Normalized Complexity-Based Ratios

Action Taken

Two normalized metrics were created:

1. word_per_section
→ Measures how much content is written per legal section
2. char_per_hearing
→ Measures case verbosity normalized by hearing duration



Rationale

Raw text counts do not consider case scope.

Normalized ratios help capture:

- How dense the petition is
- How much narrative detail is used
- Whether long texts correspond to long case durations

Normalizing text complexity produces much stronger predictive features compared to raw textual metrics.

4. Model Development Explanation

4.1 Random Forest (Baseline Model)

Why Random Forest Was Selected

Random Forest was initially used as a baseline because it is a strong general-purpose model for tabular classification. It performs well with structured data, can incorporate numerical and categorical variables along with engineered features, and naturally captures non-linear patterns. Its ensemble of decision trees also provides robustness to noise and reduces dependency on heavy preprocessing.

Problems Observed

However, the model struggled significantly in this project. The dataset contained only 55 samples but over 25 features, causing the trees to overfit and memorize the training data rather than learn general rules. The engineered features still lacked strong predictive patterns, leaving the model with insufficient signal to distinguish outcomes such as "appeal allowed" or "appeal dismissed." Additionally, Random Forest handled TF-IDF text vectors poorly because high-dimensional sparse features are incompatible with tree-based splitting.

=== CLASSIFICATION REPORT ===				
	precision	recall	f1-score	support
allowed	0.00	0.00	0.00	5
appeal allowed	0.00	0.00	0.00	2
appeal dismissed	1.00	0.17	0.29	6
petition dismissed	0.00	0.00	0.00	1
set aside	0.58	0.79	0.67	109
unknown	0.64	0.47	0.54	98
accuracy			0.60	221
macro avg	0.37	0.24	0.25	221
weighted avg	0.60	0.60	0.58	221

Outcome

Due to overfitting, weak feature signal, and ineffective use of text features, Random Forest generalized poorly and mostly defaulted to predicting the majority class.

4.2 XGBoost (Boosting Approach)

Why XGBoost Was Selected

XGBoost was chosen next because boosting often uncovers subtle patterns that bagging methods overlook. It can adjust for misclassified samples sequentially, handles class imbalance through built-in weighting, and typically achieves strong performance on engineered tabular datasets.

Problems Observed

Despite these advantages, XGBoost performed even less reliably. Boosting depends heavily on sufficient data, and on such a small dataset it amplified noise instead of true patterns. Predictions varied widely across different train-test splits, reflecting instability caused by sparse classes and dominant text features. Minority labels had too few examples for the model to form meaningful rules, so they were effectively ignored.

```
ALL DATA:
merged_outcome
set_aside_unknown    826
allowed              28
dismissed            27
Name: count, dtype: int64

TRAINING SET (allowed + dismissed):
merged_outcome
allowed            28
dismissed          27
Name: count, dtype: int64

HELD-OUT UNKNOWN SET SIZE: 826

=== CLASSIFICATION REPORT ===
              precision    recall  f1-score   support

   allowed         0.00      0.00      0.00         6
  dismissed         0.00      0.00      0.00         5
set_aside_unknown    0.94      1.00      0.97        166

   accuracy              0.94         177
  macro avg         0.31      0.33      0.32         177
 weighted avg         0.88      0.94      0.91         177
...
```

Outcome

XGBoost failed to surpass the baseline and leaned heavily toward the majority class, indicating that boosting techniques were not suitable for the dataset size and distribution.

4.3 Linear Models (Final Attempt)

Why Linear Models Were Selected

Linear models were explored next because they work well with TF-IDF representations, handle high-dimensional sparse vectors more effectively than trees, and reduce overfitting by assuming simpler decision boundaries. They also offer clearer interpretability of feature influence.

Problems Observed

However, linear classifiers require sufficient class examples to separate classes effectively. With extremely small and imbalanced class distributions, they could not establish distinct boundaries in the feature space. The TF-IDF vectors further contributed extreme sparsity, making generalization difficult.

```
Data shape after filtering: (55, 27)
outcome
appeal dismissed    24
allowed            18
appeal allowed      10
petition dismissed   3
Name: count, dtype: int64

=== CLASSIFICATION REPORT ===
              precision    recall  f1-score   support

    allowed           0.00      0.00      0.00         3
  appeal allowed      0.00      0.00      0.00         2
  appeal dismissed    0.45      1.00      0.62         5
petition dismissed    0.00      0.00      0.00         1

   accuracy           0.45      0.45      0.45        11
  macro avg           0.11      0.25      0.16        11
 weighted avg          0.21      0.45      0.28        11

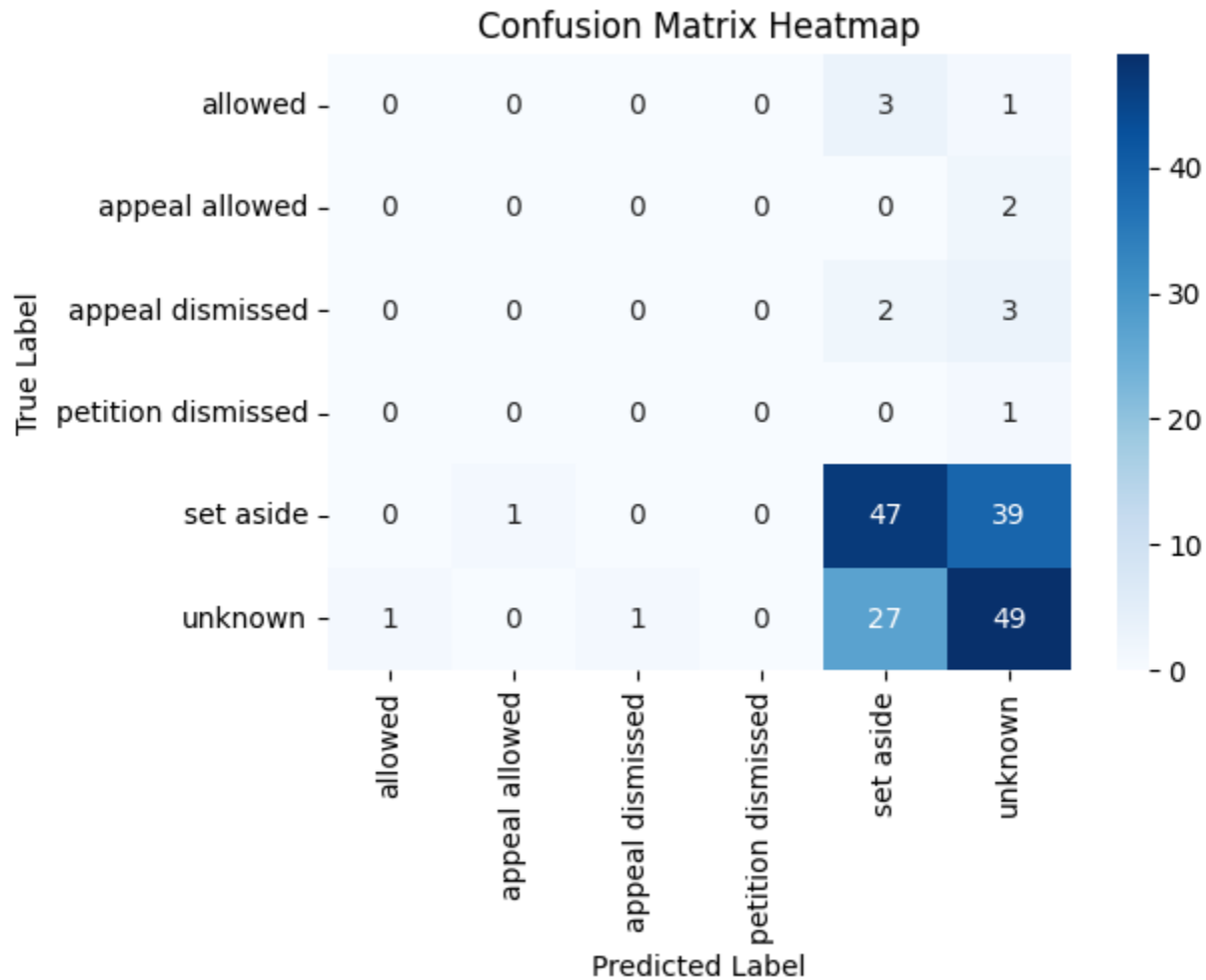
=== CONFUSION MATRIX ===
[[0 0 3 0]
 [0 0 2 0]
 [0 0 5 0]
 [0 0 1 0]]
```

Outcome

The linear model defaulted to predicting the majority label (“appeal dismissed”), achieving around 45% accuracy—simply matching the proportion of the dominant class rather than demonstrating meaningful learning.

6.VISUALIZING MODEL RESULTS:

Confusion Matrix Heatmap:



A confusion matrix heatmap was plotted to visually assess how accurately the model distinguishes between the two decision outcomes (“allowed” and “dismissed”). The diagonal cells represent correct predictions, while off-diagonal cells show where the model misclassified a case. The color gradient helps highlight class imbalance and shows whether the classifier tends to favor one class over the other.

7. Conclusion

Overall, the project demonstrates a complete pipeline for transforming unstructured judicial documents into a structured dataset, enriching it through feature engineering, and applying multiple machine learning approaches for outcome prediction. The experimental results highlight important insights regarding the nature of legal data and the requirements for achieving reliable predictive performance.

While the developed models provide an initial benchmark, the analysis indicates that meaningful predictive accuracy in this domain depends on access to larger and more balanced datasets. The current dataset, although valuable for exploratory analysis and methodological development, contains limited samples per outcome category, which restricts the ability of classification algorithms to learn stable decision boundaries.

Future work can build upon the foundation established in this project by:

- Expanding the dataset to include a higher volume of judgments across different case categories.
- Ensuring more balanced class representation to support multi-class predictive modeling.
- Incorporating enhanced text-preprocessing pipelines and legal-domain embeddings to better capture semantic patterns.
- Exploring transformer-based architectures once sufficient data is available to support deep learning methods.

This project provides a solid starting point for computational legal analytics and sets the groundwork for more advanced modeling techniques as richer datasets become available